

ODSC AI EAST 2026

AI ACCELERATOR

INTRODUCTION TO THE MATH BEHIND TRANSFORMERS AND LLMS

DAVID HOYLE, PHD

Research Data Science Specialist

dunnhumby



Outline

github.com/dchoyle/ODSCEast2026_TransformerBootcamp

- | | |
|----|----------------------------------|
| 01 | What we will learn |
| 02 | Predicting the next word |
| 03 | Simple approaches |
| 04 | Word embeddings |
| 05 | Transformers |
| 06 | Back to predicting the next word |
| 07 | Different types of transformers |
| 08 | Wrap-up |



Outline

github.com/dchoyle/ODSCEast2026_TransformerBootcamp

- | | |
|----|----------------------------------|
| 01 | What we will learn |
| 02 | Predicting the next word |
| 03 | Simple approaches |
| 04 | Word embeddings |
| 05 | Transformers |
| 06 | Back to predicting the next word |
| 07 | Different types of transformers |
| 08 | Wrap-up |

Break



Outline

github.com/dchoyle/ODSCEast2026_TransformerBootcamp

- | | |
|----|----------------------------------|
| 01 | What we will learn |
| 02 | Predicting the next word |
| 03 | Simple approaches |
| 04 | Word embeddings |
| 05 | Transformers |
| 06 | Back to predicting the next word |
| 07 | Different types of transformers |
| 08 | Wrap-up |



Code example

Code example

What we will learn

- The mathematical challenges of next word prediction
- **The basic mathematical ideas behind Transformers**
- How Transformers predict words and sentences
- **To get comfortable at a high-level with how LLMs work at a mathematical level**
- To lose your fear of the science of LLMs



What we will **not** learn

- How to train foundational models
- How to build transformer-based chatbots
- How to prompt
- Agentic AI
- How to make \$\$\$Millions from AI



What I assume you know

- Basic probability, e.g. conditional probability
- Basic linear algebra, e.g. matrix multiplication as a transformation
- Basic machine learning, e.g. training a model by minimizing a loss function



Predicting the next word

What this enables, tokens and sequences

Predicting the next word

- What are we trying to achieve?
 - The cat sat on the

Predicting the next word

- What are we trying to achieve?
 - The cat sat on the **mat**

Predicting the next word

- What are we trying to achieve?
 - The quick brown fox jumped over the

Predicting the next word

- What are we trying to achieve?
 - The quick brown fox jumped over the lazy
 - The quick brown fox jumped over the lazy

Predicting the next word

- What are we trying to achieve?
 - The quick brown fox jumped over the lazy
 - The quick brown fox jumped over the lazy dog

Predicting the next word

- What are we trying to achieve?

$$\begin{aligned} w_1, w_2, \dots, w_K &\rightarrow w_1, w_2, \dots, w_K, w_{K+1} \rightarrow w_1, w_2, \dots, w_K, w_{K+1}, w_{K+2} \\ &\rightarrow w_1, w_2, \dots, w_K, w_{K+1}, \dots, w_{2K} \end{aligned}$$

Predicting the next word

- What are we trying to achieve?

“I wandered lonely as a cloud that floats on high o'er vales and hills,
when all at once

→ “I wandered lonely as a cloud that floats on high o'er vales and
hills, when all at once **I saw a crowd, a host of golden daffodils**”

William Wordsworth, 1807

Words as tokens

- Words are the sentence natural units of a sentence.
- We breaks down a sentence into these units:
the cat sat on the mat → [the, cat, sat, on, the mat]
- We call these units “tokens”
- Tokens don’t have to be complete words:
quick brown fox jumped over → [quick, brown, fox, jump, ed, over]

Sequences

- A natural language sentence is just a sequence of tokens
- Our modelling challenge is “predict the next token in given the preceding sequence of tokens”
- We find sequences of tokens everywhere, not just in language
- I will almost always be referring to words in a sentence

Simple approaches

Naïve models that are wrong and what we can learn from that

Simple approaches: Markov models

Example: “The more you know, the more you realize you don't know”

Aristotle

$P_{ij} = \text{Prob}(\text{Word } i \text{ in vocabulary} \mid \text{preceding word is word } j \text{ in vocabulary})$

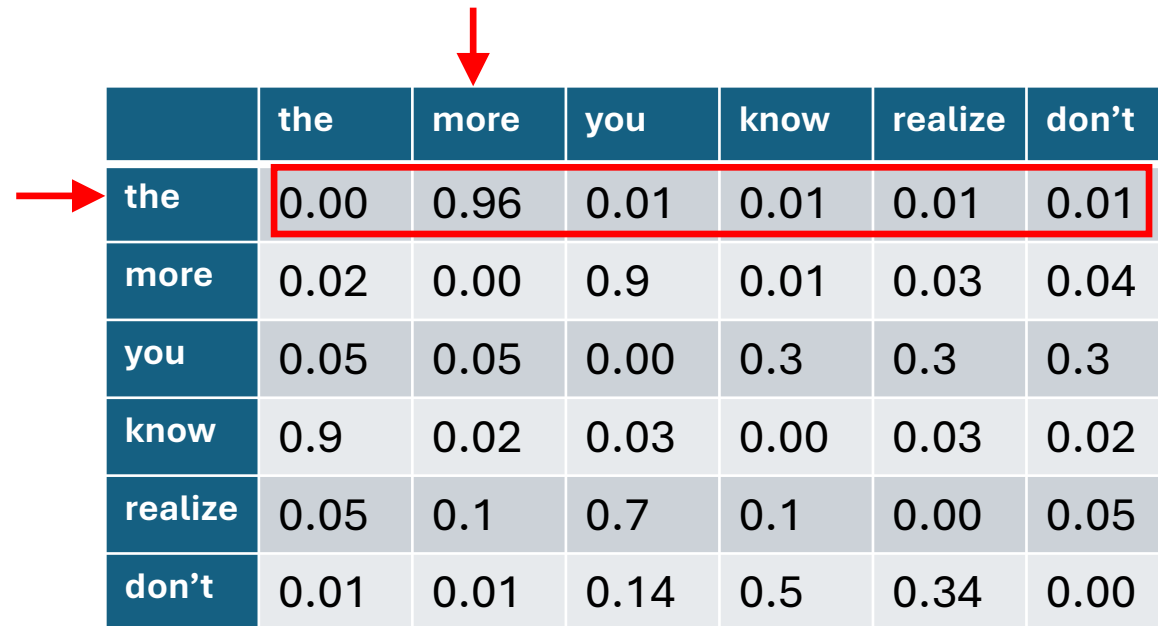
$P_{ij} =$ From

	To					
	the	more	you	know	realize	don't
the	0.00	0.96	0.01	0.01	0.01	0.01
more	0.02	0.00	0.9	0.01	0.03	0.04
you	0.05	0.05	0.00	0.3	0.3	0.3
know	0.9	0.02	0.03	0.00	0.03	0.02
realize	0.05	0.1	0.7	0.1	0.00	0.05
don't	0.01	0.01	0.14	0.5	0.34	0.00

Simple approaches: Markov models

Example: “The more you know, the more you realize you don't know”

Aristotle



	the	more	you	know	realize	don't
the	0.00	0.96	0.01	0.01	0.01	0.01
more	0.02	0.00	0.9	0.01	0.03	0.04
you	0.05	0.05	0.00	0.3	0.3	0.3
know	0.9	0.02	0.03	0.00	0.03	0.02
realize	0.05	0.1	0.7	0.1	0.00	0.05
don't	0.01	0.01	0.14	0.5	0.34	0.00

$$P_{ij} =$$

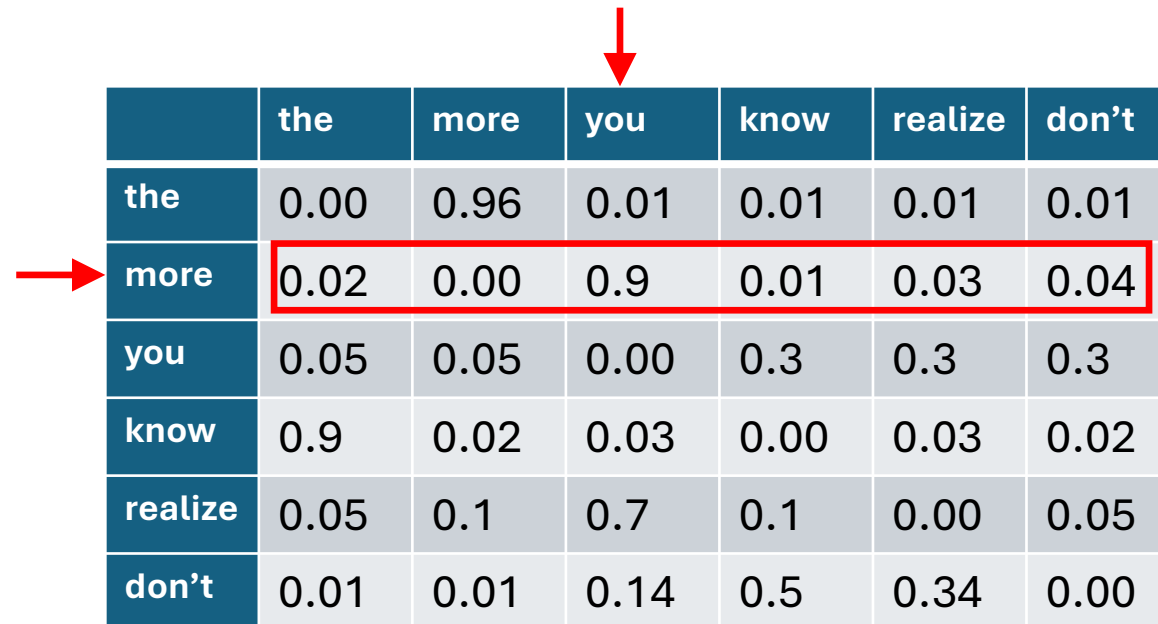
The more
i *j*

Simple approaches: Markov models

Example: “The more you know, the more you realize you don't know”

Aristotle

$$P_{ij} =$$



	the	more	you	know	realize	don't
the	0.00	0.96	0.01	0.01	0.01	0.01
more	0.02	0.00	0.9	0.01	0.03	0.04
you	0.05	0.05	0.00	0.3	0.3	0.3
know	0.9	0.02	0.03	0.00	0.03	0.02
realize	0.05	0.1	0.7	0.1	0.00	0.05
don't	0.01	0.01	0.14	0.5	0.34	0.00

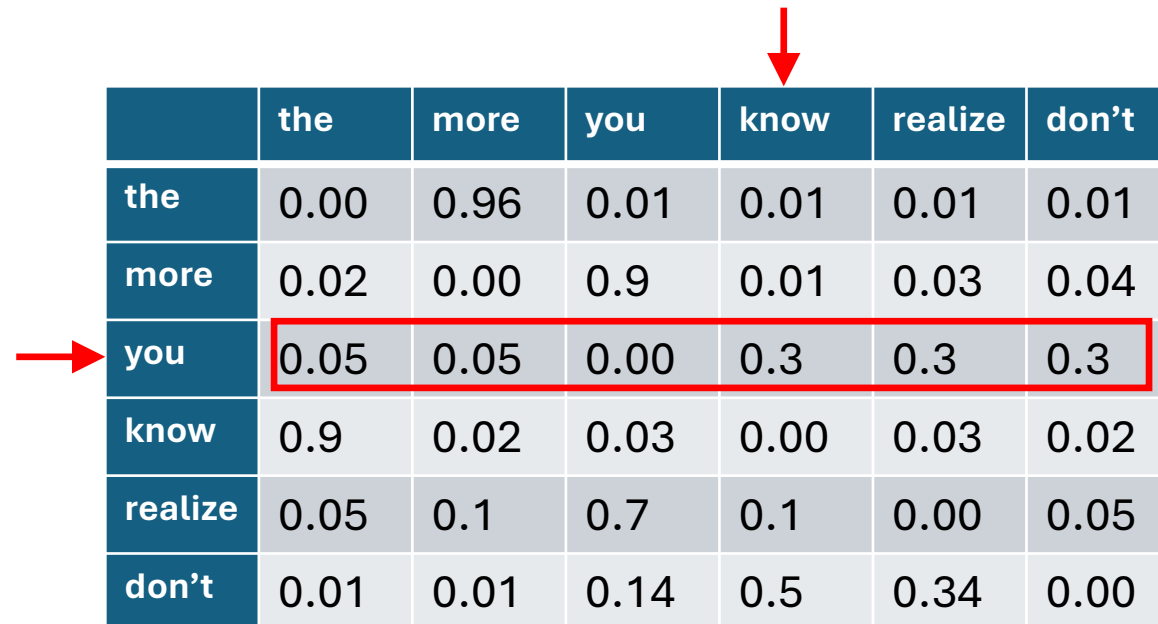
The more you
i *j*

Simple approaches: Markov models

Example: “The more you know, the more you realize you don't know”

Aristotle

$P_{ij} =$



	the	more	you	know	realize	don't
the	0.00	0.96	0.01	0.01	0.01	0.01
more	0.02	0.00	0.9	0.01	0.03	0.04
you	0.05	0.05	0.00	0.3	0.3	0.3
know	0.9	0.02	0.03	0.00	0.03	0.02
realize	0.05	0.1	0.7	0.1	0.00	0.05
don't	0.01	0.01	0.14	0.5	0.34	0.00

The more you know

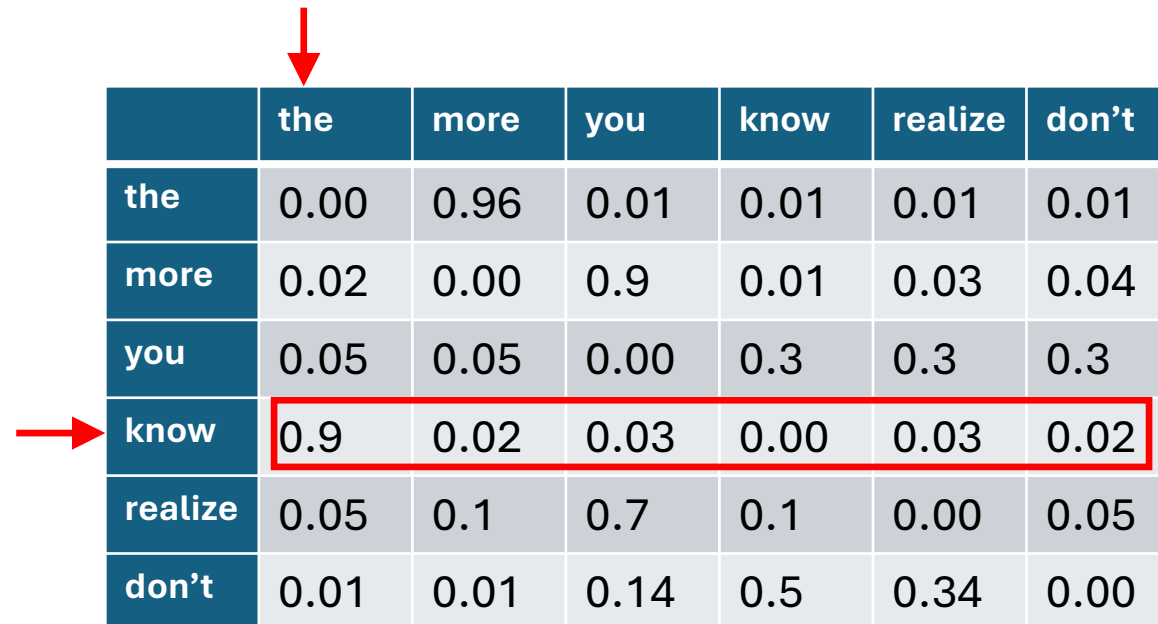
$i \quad j$

Simple approaches: Markov models

Example: “The more you know, the more you realize you don't know”

Aristotle

$P_{ij} =$



	the	more	you	know	realize	don't
the	0.00	0.96	0.01	0.01	0.01	0.01
more	0.02	0.00	0.9	0.01	0.03	0.04
you	0.05	0.05	0.00	0.3	0.3	0.3
know	0.9	0.02	0.03	0.00	0.03	0.02
realize	0.05	0.1	0.7	0.1	0.00	0.05
don't	0.01	0.01	0.14	0.5	0.34	0.00

The more you know the

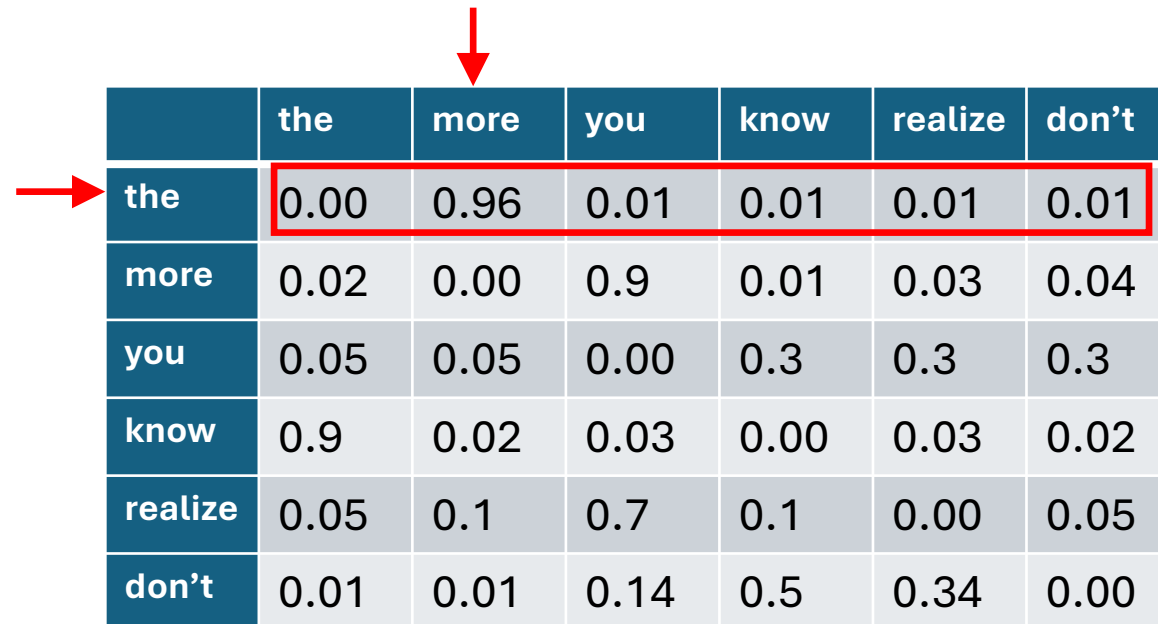
$i \quad j$

Simple approaches: Markov models

Example: “The more you know, the more you realize you don't know”

Aristotle

$P_{ij} =$



	the	more	you	know	realize	don't
the	0.00	0.96	0.01	0.01	0.01	0.01
more	0.02	0.00	0.9	0.01	0.03	0.04
you	0.05	0.05	0.00	0.3	0.3	0.3
know	0.9	0.02	0.03	0.00	0.03	0.02
realize	0.05	0.1	0.7	0.1	0.00	0.05
don't	0.01	0.01	0.14	0.5	0.34	0.00

The more you know the more

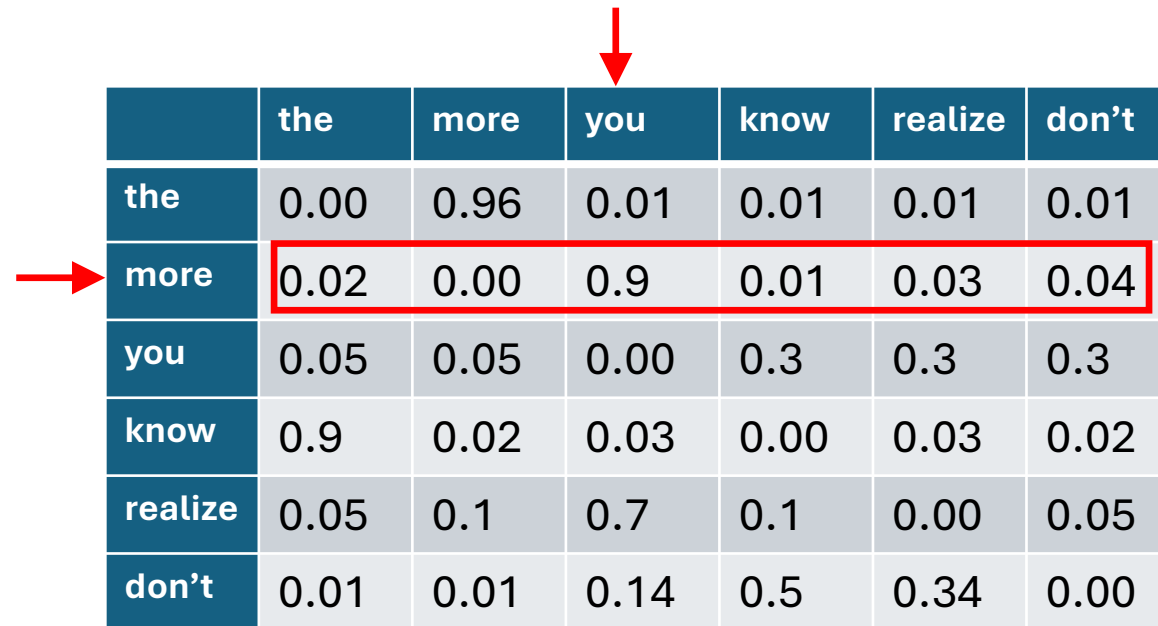
$i \quad j$

Simple approaches: Markov models

Example: “The more you know, the more you realize you don't know”

Aristotle

$P_{ij} =$



	the	more	you	know	realize	don't
the	0.00	0.96	0.01	0.01	0.01	0.01
more	0.02	0.00	0.9	0.01	0.03	0.04
you	0.05	0.05	0.00	0.3	0.3	0.3
know	0.9	0.02	0.03	0.00	0.03	0.02
realize	0.05	0.1	0.7	0.1	0.00	0.05
don't	0.01	0.01	0.14	0.5	0.34	0.00

The more you know the more you

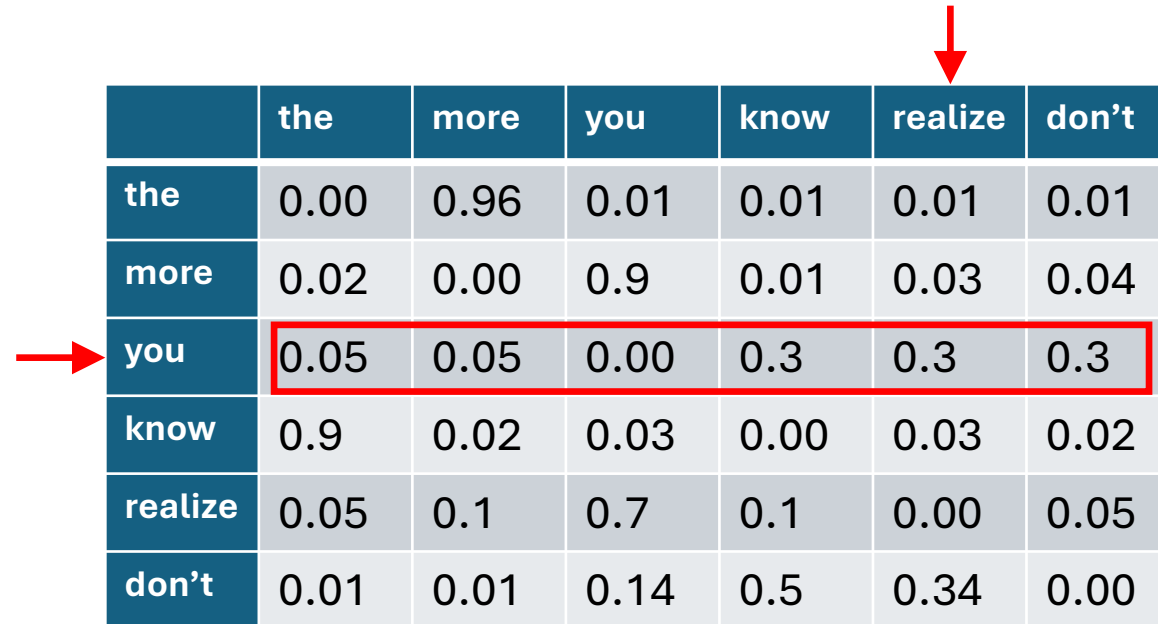
$i \quad j$

Simple approaches: Markov models

Example: “The more you know, the more you realize you don't know”

Aristotle

$P_{ij} =$



	the	more	you	know	realize	don't
the	0.00	0.96	0.01	0.01	0.01	0.01
more	0.02	0.00	0.9	0.01	0.03	0.04
you	0.05	0.05	0.00	0.3	0.3	0.3
know	0.9	0.02	0.03	0.00	0.03	0.02
realize	0.05	0.1	0.7	0.1	0.00	0.05
don't	0.01	0.01	0.14	0.5	0.34	0.00

The more you know the more you realize

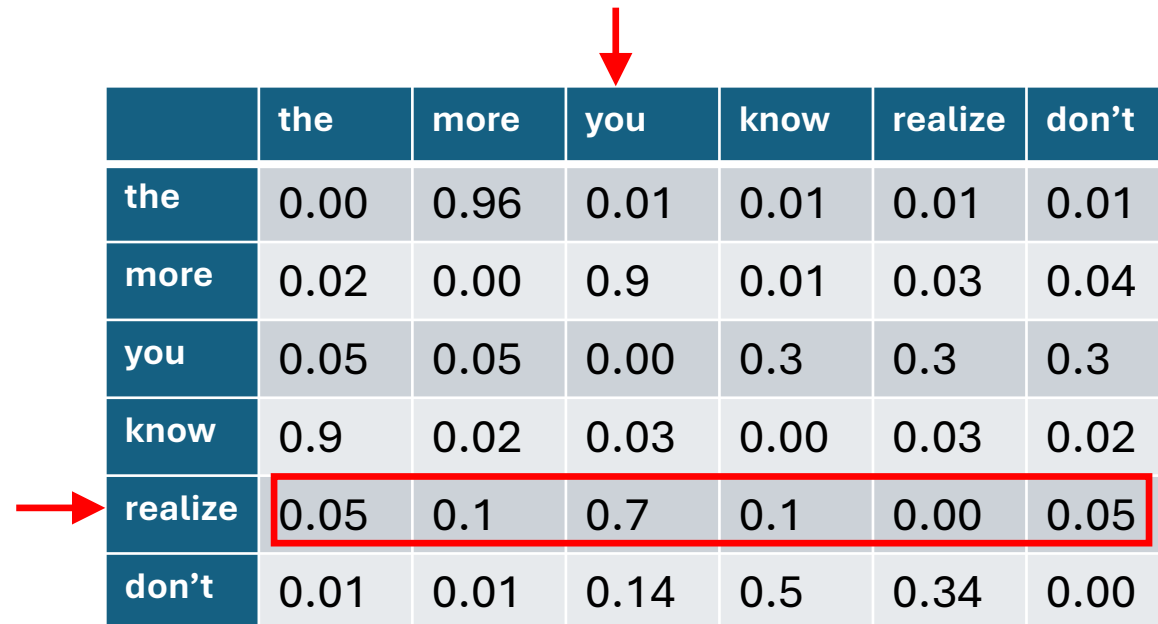
$i \quad j$

Simple approaches: Markov models

Example: “The more you know, the more you realize you don't know”

Aristotle

$P_{ij} =$



	the	more	you	know	realize	don't
the	0.00	0.96	0.01	0.01	0.01	0.01
more	0.02	0.00	0.9	0.01	0.03	0.04
you	0.05	0.05	0.00	0.3	0.3	0.3
know	0.9	0.02	0.03	0.00	0.03	0.02
realize	0.05	0.1	0.7	0.1	0.00	0.05
don't	0.01	0.01	0.14	0.5	0.34	0.00

The more you know the more you realize you

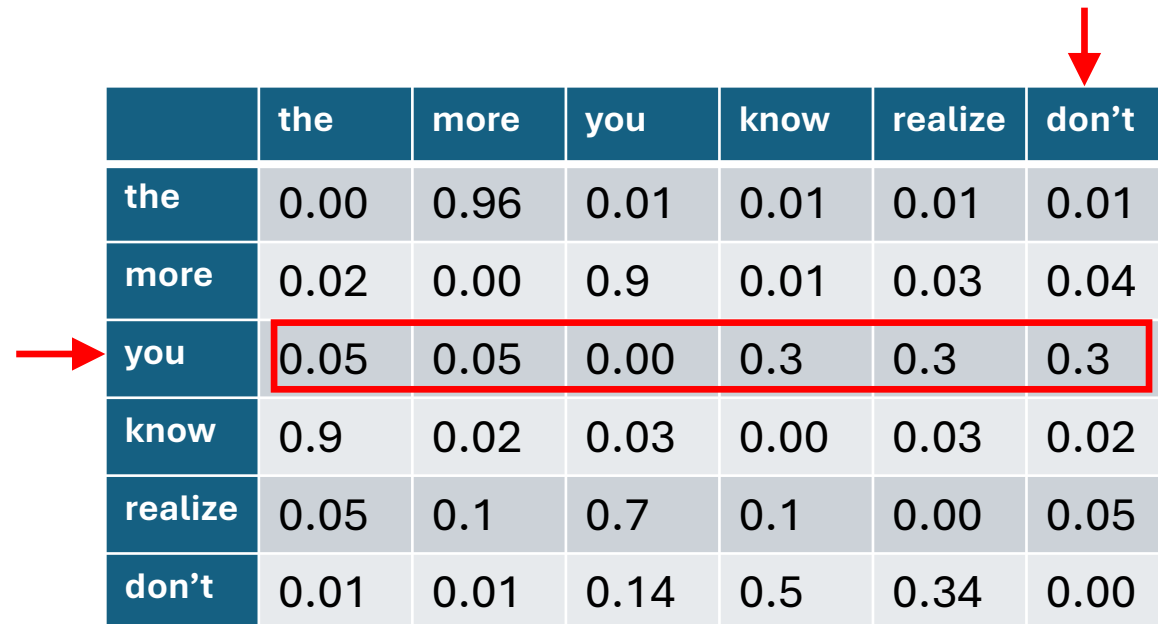
$i \quad j$

Simple approaches: Markov models

Example: “The more you know, the more you realize you don't know”

Aristotle

$P_{ij} =$



	the	more	you	know	realize	don't
the	0.00	0.96	0.01	0.01	0.01	0.01
more	0.02	0.00	0.9	0.01	0.03	0.04
you	0.05	0.05	0.00	0.3	0.3	0.3
know	0.9	0.02	0.03	0.00	0.03	0.02
realize	0.05	0.1	0.7	0.1	0.00	0.05
don't	0.01	0.01	0.14	0.5	0.34	0.00

The more you know the more you realize you don't

$i \quad j$

Simple approaches: Markov models

Example: “The more you know, the more you realize you don't know”

Aristotle

$P_{ij} =$



	the	more	you	know	realize	don't
the	0.00	0.96	0.01	0.01	0.01	0.01
more	0.02	0.00	0.9	0.01	0.03	0.04
you	0.05	0.05	0.00	0.3	0.3	0.3
know	0.9	0.02	0.03	0.00	0.03	0.02
realize	0.05	0.1	0.7	0.1	0.00	0.05
don't	0.01	0.01	0.14	0.5	0.34	0.00



The more you know the more you realize you don't know

$i \quad j$

Simple approaches: Markov models

- Each separate word corresponds to a column in the transition matrix:
“know” = (0,0,0,1,0,0)
- Small fixed vocabulary $\Rightarrow$ Moving six-dimensional discrete state-space
- Model is probabilistic:
 $P(\text{“know”} \mid \text{“you”}) = 0.3$, $P(\text{“realize”} \mid \text{“you”}) = 0.3$, $P(\text{“don’t”} \mid \text{“you”}) = 0.3$

Could easily have generated,

‘The more you don’t the more you realize you don’t realize’

Simple approaches: Markov models

Problems:

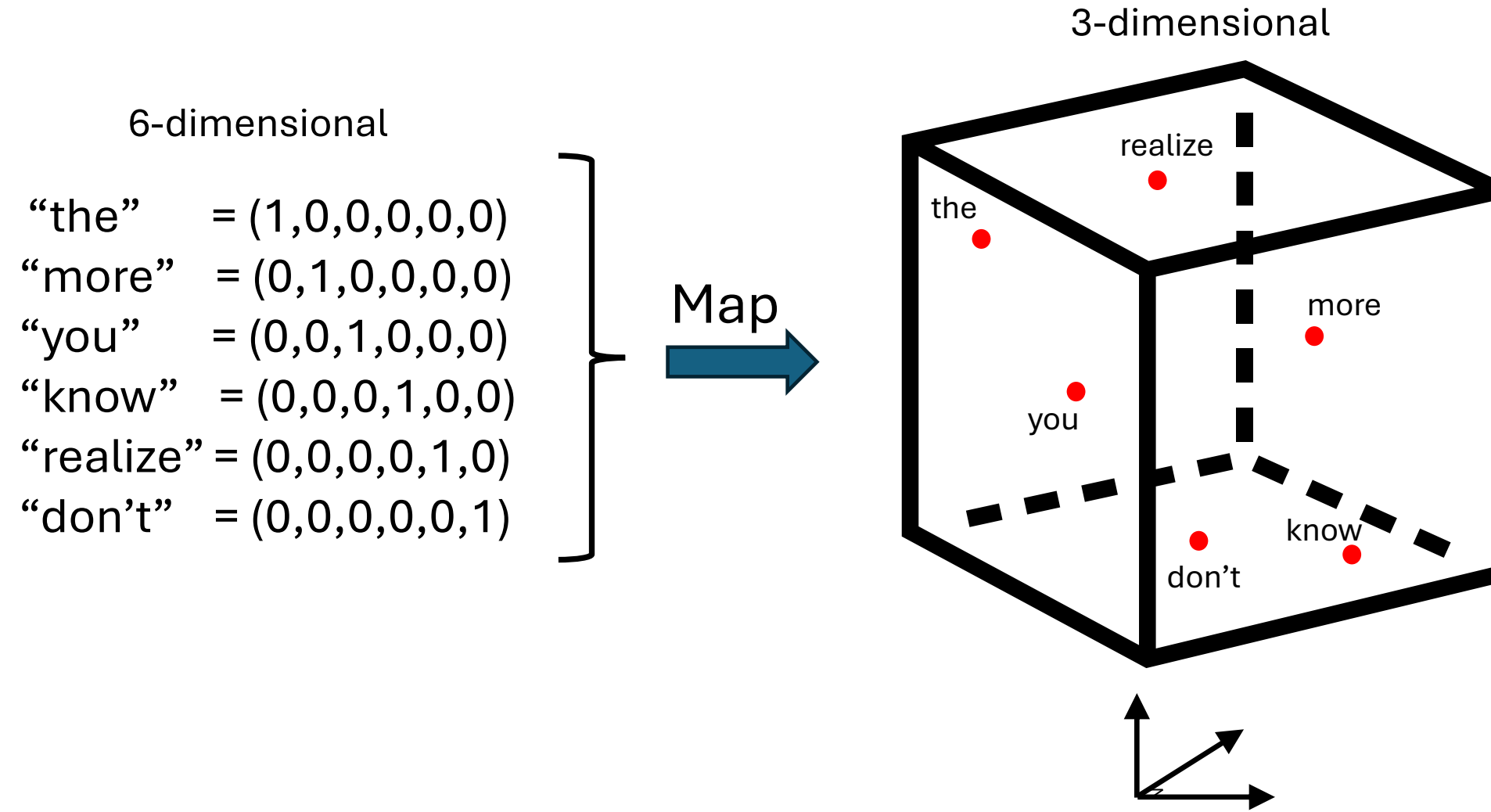
1. Our vocabulary is large, e.g. 170000 – 700000 words
→ Large transition matrix
- 2a. We only take into account the preceding word
→ “Many adults drink beer” , “Many newborn babies drink beer”
“beer” has the same probability of occurring after the word “drink” in this model
- 2b. Expanding to transitions between pairs of words increases state space
e.g. (many, adults) → (adults, drink) → (drink, beer)

Word embeddings

Correcting our mistakes

Word embeddings

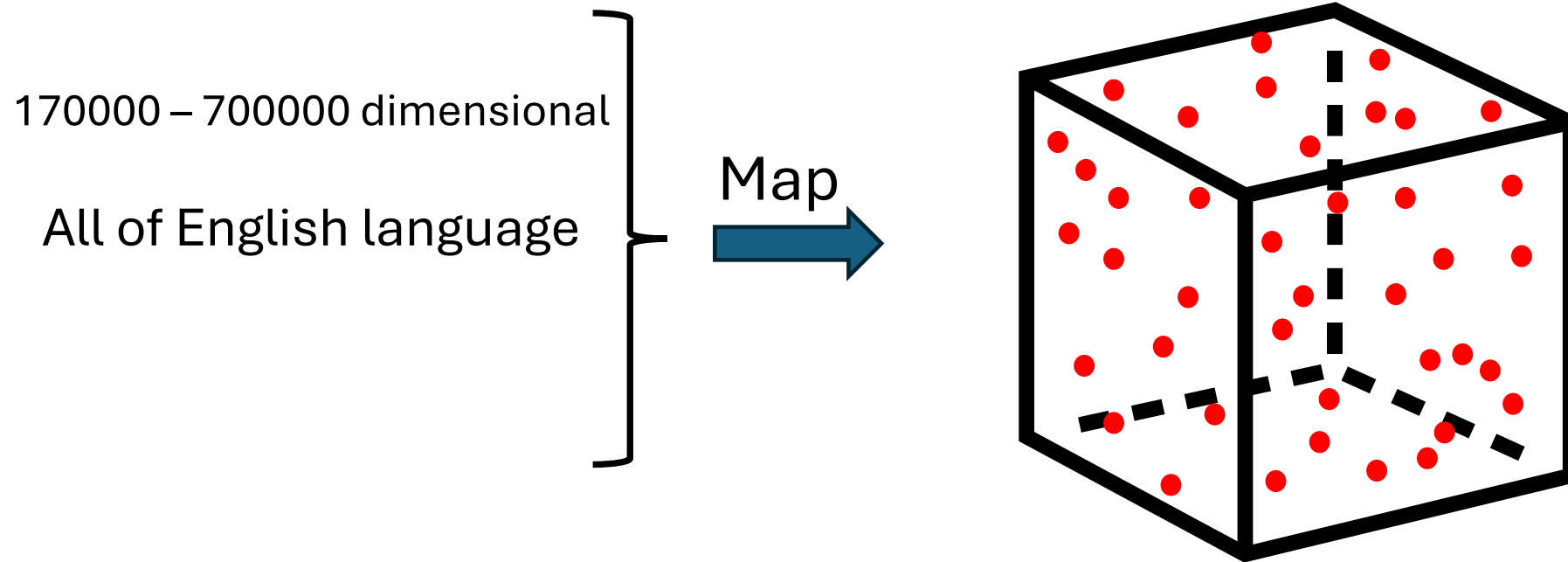
- Reduce to a fixed lower dimensionality



Word embeddings

- Reduce to a fixed lower dimensionality

Word2Vec: 300-dimensional
GPT-3 (Davinci): 12288-dimensional
LlaMA 3.1-70B: 8192-dimensional
GPT-4.5: 3072-dimensional



Word embeddings

Tokens



Index	the	cat	sat	on	The	mat
1	2.4249×10^{-2}	4.7426×10^{-2}	-8.1093×10^{-2}	-2.0204×10^{-2}	2.4249×10^{-2}	-7.3271×10^{-2}
2	4.8235×10^{-3}	-4.2203×10^{-2}	-6.3660×10^{-2}	-6.6599×10^{-2}	4.8235×10^{-3}	-8.9358×10^{-2}
3	1.8411×10^{-2}	2.8491×10^{-2}	3.6398×10^{-2}	2.5621×10^{-2}	1.8411×10^{-2}	1.0698×10^{-1}
4	1.1867×10^{-2}	-4.4481×10^{-2}	3.3358×10^{-2}	1.7926×10^{-2}	1.1867×10^{-2}	1.1881×10^{-1}
5	1.9167×10^{-2}	-2.6467×10^{-2}	1.1537×10^{-1}	8.4262×10^{-23}	1.9167×10^{-2}	5.6133×10^{-3}
⋮	⋮	⋮	⋮	⋮	⋮	⋮
⋮	⋮	⋮	⋮	⋮	⋮	⋮
300	3.4856×10^{-3}	1.3847×10^{-1}	9.3088×10^{-2}	5.2301×10^{-3}		1.8160×10^{-1}

Embedding
vector

dimension d



Code example

Working with word embeddings

Code Example 1

- Open up the Jupyter notebook ODSCEast2026_CodeExample1.ipynb in the github repository

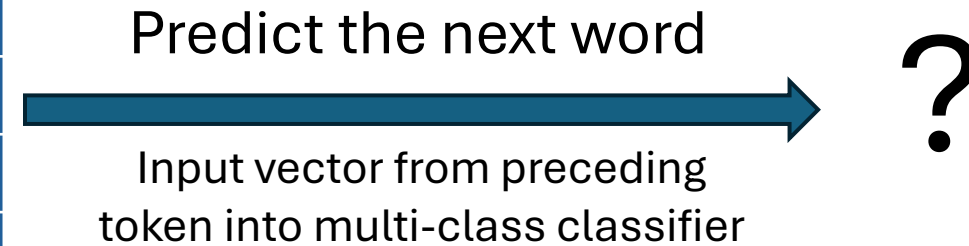
https://github.com/dchoyle/ODSCEast2026_TransformerBootcamp/

Transformers

Creating context aware embeddings

Transforming embeddings

the	cat	sat	on	the



- Each token knows nothing about the broader context of the preceding words
- Need new vectors for each token that take into account the preceding context
 - We will make a Transformer
- For now, create new vectors that take into account the context of the whole sentence

Attention

Input embeddings

the	cat	sat	on	the



Updated embeddings

the	cat	sat	on	the

- New vector for token n = weighted average across all vectors in sentence

$$\underline{y}_n = \sum_{n'} \underline{y}_{n'} A_{n,n'} \quad , \quad \sum_{n'} A_{n,n'} = 1 \quad \forall n$$

- Actually, let's take weighted average of vectors that we'll learn as part of the training process

$$\underline{y}_n = \sum_{n'} \underline{v}_{n'} A_{n,n'} \quad , \quad \underline{v}_n^\top = \underline{y}_n^\top \underline{U}_V$$

Self-Attention

- Where do we get $A_{n,n'}$ from?
 - 1) Needs to depend upon vectors for tokens n and n'
 - 2) Needs to reflect similarity between vectors for tokens n and n'

$$\Rightarrow A_{n,n'} \sim \underline{y}_n \cdot \underline{y}_{n'} = \underline{y}_n^\top \underline{y}_{n'}$$

Not a probability !

- Not a problem – exponentiate and normalize

$$A_{n,n'} = \frac{\exp(\underline{y}_n \cdot \underline{y}_{n'})}{\sum_{n''} \exp(\underline{y}_n \cdot \underline{y}_{n''})} \quad \text{just softmax function applied to } \underline{y}_n \cdot \underline{y}_{n'}$$

Self-Attention

- Are $\underline{y}_n$ and $\underline{y}_{n'}$ the best vectors to measure similarity between tokens n and n' ?
- Ok, let's allow for a learnable linear transformation

$$\underline{y}_n \rightarrow \underline{\underline{U}} \underline{y}_n = \underline{z}_n$$

- Similarity between tokens is now defined as,

$$\underline{z}_n \cdot \underline{z}_{n'} = \underline{y}_n^\top \underline{\underline{U}}^\top \underline{\underline{U}} \underline{y}_{n'}$$

- Vector $\underline{y}_n$ is mapped to a new vector space to get $\underline{z}_n$ and using $\underline{z}_n$ to pick out which other token vectors are most similar in this new space
 - $\underline{z}_n$ is like a **query** vector which is matched against **key** vectors $\underline{z}_{n'}$

Self-Attention

- We can add more flexibility to model.
 - Different linear mappings to create the query vectors and to create the key vectors

$$\text{query vector} = \underline{\underline{U}}_q \underline{y}$$

$$\underline{\underline{U}}_q, \underline{\underline{U}}_k = d \times d_k \quad , \quad d_k < d$$

$$\text{key vector} = \underline{\underline{U}}_k \underline{y}$$

- Similarity now defined as, query vector $\cdot$ key vector
- Similarity between token n and n' now given by

$$\underline{y}_n^\top \underline{\underline{U}}_q^\top \underline{\underline{U}}_k \underline{y}_{n'}$$

Self-Attention

- We can add more flexibility to model.
 - Different linear mappings to create the query vectors and to create the key vectors

$$\text{query vector} = \underline{\underline{U}}_q \underline{y}$$

$$\underline{\underline{U}}_q, \underline{\underline{U}}_k = d \times d_k \quad , \quad d_k < d$$

$$\text{key vector} = \underline{\underline{U}}_k \underline{y}$$

- Similarity now defined as, query vector $\cdot$ key vector
- Similarity between token n and n' now given by

$$\frac{\underline{y}_n^\top \underline{\underline{U}}_q^\top \underline{\underline{U}}_k \underline{y}_{n'}}{\sqrt{d_k}}$$

Self-Attention

- Putting it altogether (almost)

$$\text{New embedding vector for } n = \sum_{n'} v_{n'} \frac{\exp(\underline{y}_n^\top \underline{U}_q^\top \underline{U}_k \underline{y}_{n'})}{\sum_{n''} \exp(\underline{y}_n^\top \underline{U}_q^\top \underline{U}_k \underline{y}_{n''})}$$

- In English:

New embedding vector for token n = weighted average of the vectors $\underline{v}_{n'}$

$\text{weight}(n, n') = \text{Softmax}(\text{similarity between query vector for } n \text{ and key vector for } n')$

We learn the vectors $\underline{v}_n$ and the projection matrices $\underline{U}_q$, $\underline{U}_k$ through training

Attention: Masking

- When we are building a model to predict the next word we only want to take into account the preceding context

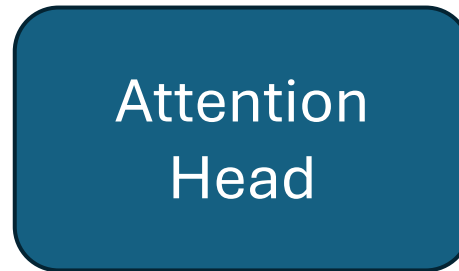
- We want $A_{n,n'} = 0$ if $n' > n$

	the	cat	sat	on	the
the	1.0	0	0	0	0
cat	0.2	0.8	0	0	0
sat	0.35	0.15	0.5	0	0
on	0.08	0.12	0.34	0.46	0
the	0.2	0.25	0.30	0.05	0.2

- We can easily achieve this by putting correspond ‘similarity’ values equal to $-\infty$ before applying softmax operation

Attention Head

Input embedding vector for token

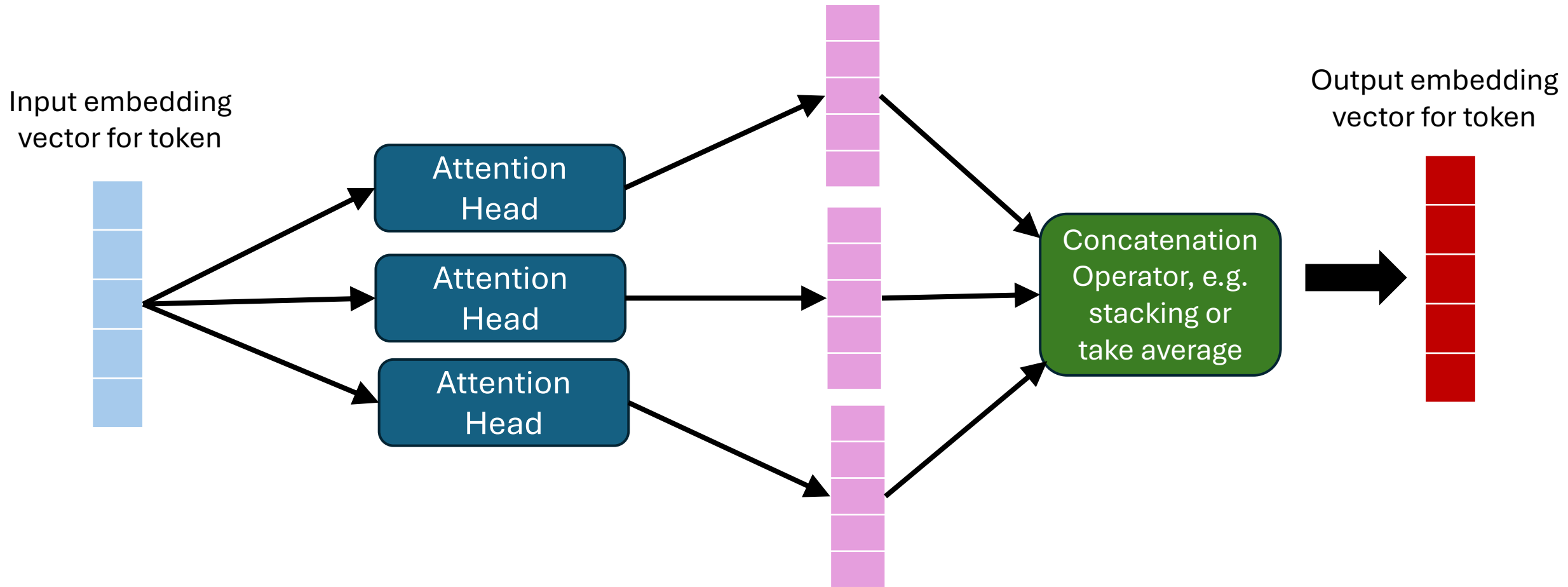


Output embedding vector for token



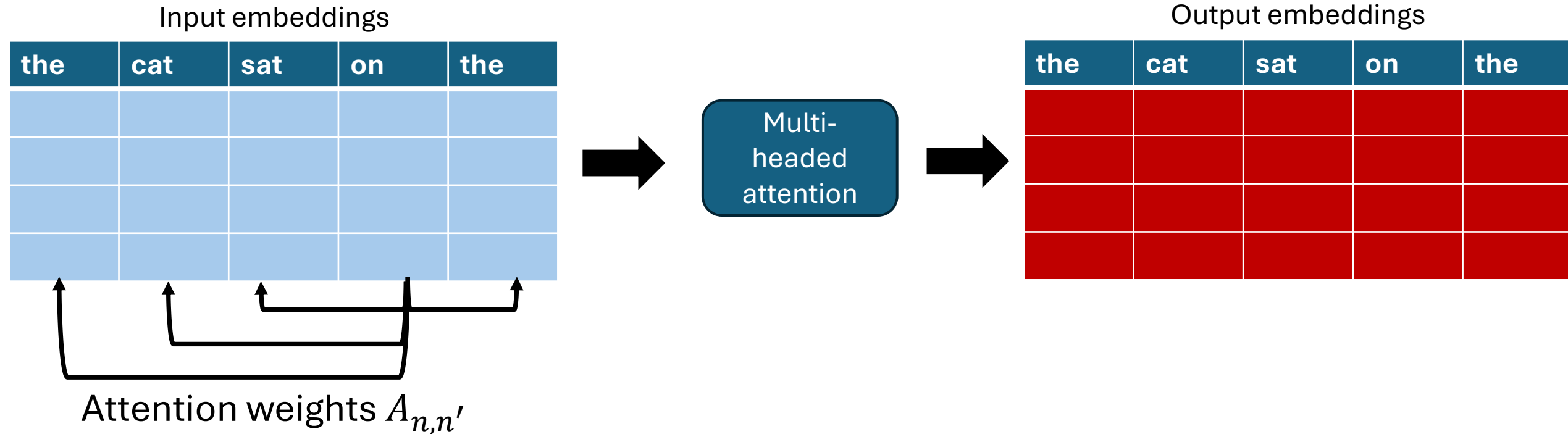
- We can wrap all the mathematics of self-attention into an operation we will call an Attention Head
- An attention head maps an embedding vector to an updated embedding vector taking into account what the surrounding words (tokens) are
- Problem: We use the same attention weight $A_{n,n'}$ for each dimension of the embedding space
We can't two tokens similar in one dimension and very dis-similar in another dimension

Multi-headed Self-Attention



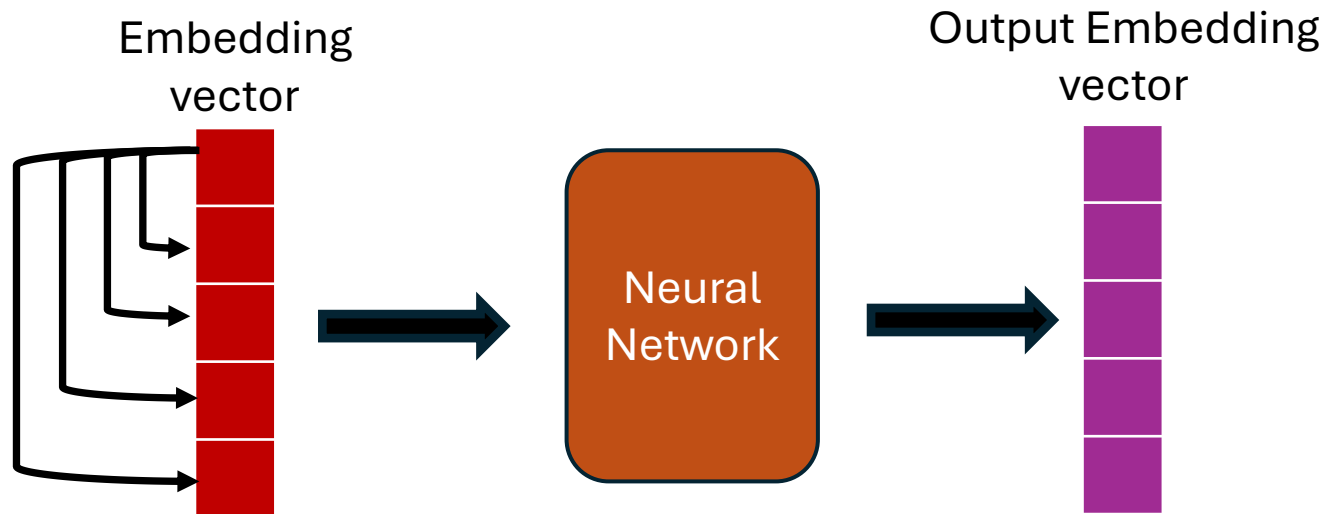
- We solve this by having multiple (H) self-attention heads.
- We combine the output vectors of all the attention heads into a single vector
- Typically, we make output dimension of each attention head d/H

Neural Network Sub-Layer



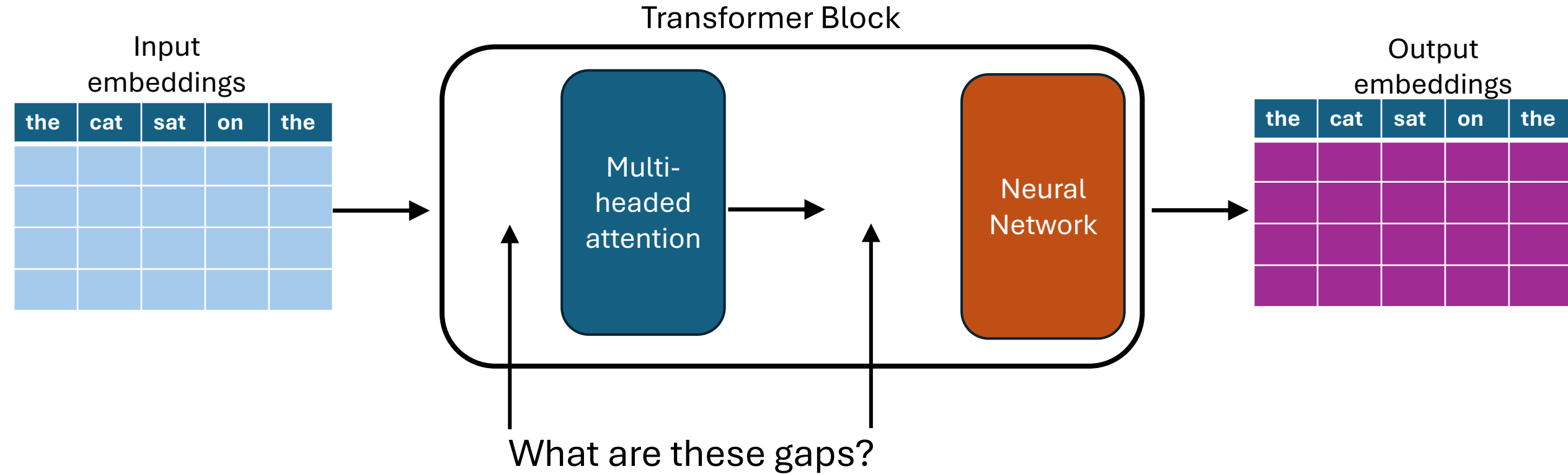
- Attention process is column-wise (token-to-token)
- We haven't allowed the different rows within a column to interact when producing the updated embedding

Neural Network Sub-Layer



Parameters of neural network are learnt as part of the overall model training

The Transformer Block

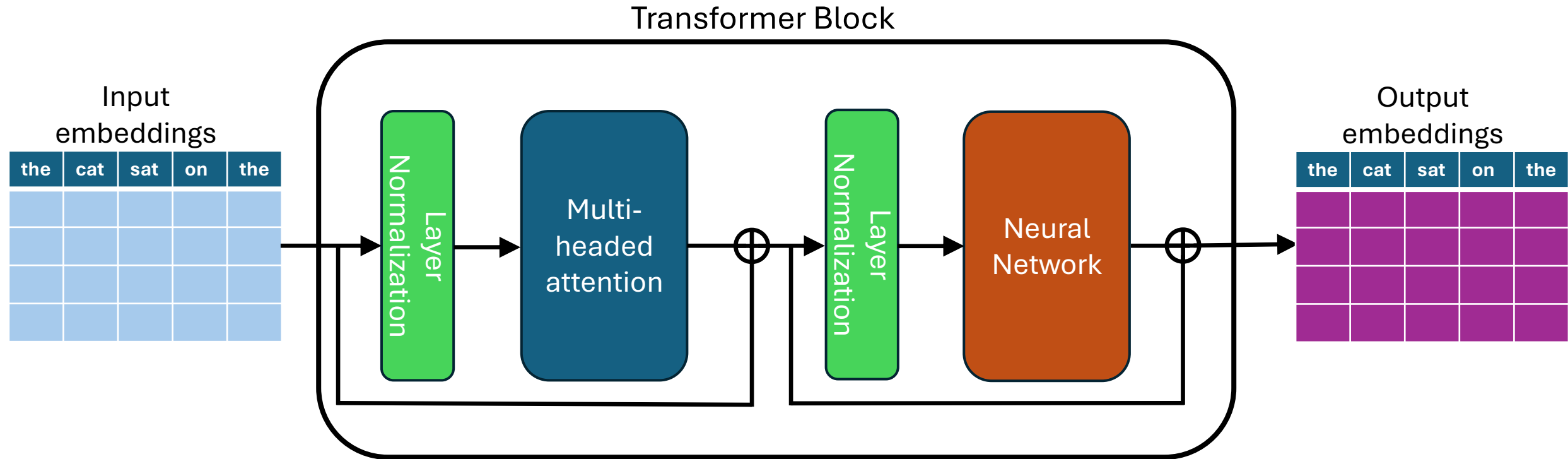


Technical transformations

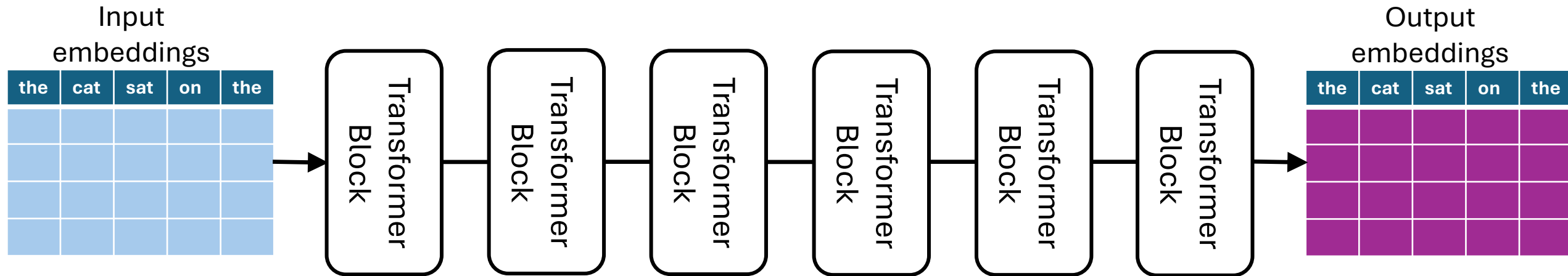
- We apply two main additional transformations to the data to improve efficiency and stability of the training process:
 - 1) Layer-wise normalization – apply a standardization (location and scale) transformation before each sub-layer
 - 2) We use the multi-headed attention and neural network transformations to model the residuals of the input-output relations

Output vector = Input vector + Transformation(Input vector)

The Transformer Block



The Transformer Model

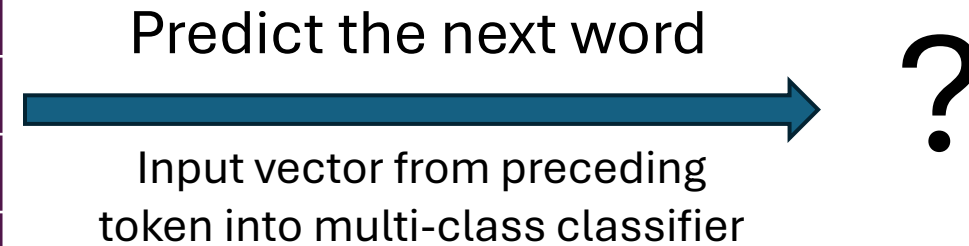


Back to predicting the next word

Back to our goal

Predicting the next word from context embeddings

the	cat	sat	on	the



- Use softmax to compute probabilities of each word in our vocabulary given the context embedding vector $\underline{f}$
- $$P(\text{word } k \mid \text{context embedding vector } \underline{f}) = \frac{\exp(\underline{g}_k^\top \underline{f})}{\sum_{k'} \exp(\underline{g}_{k'}^\top \underline{f})}$$
- We learn the vectors $\underline{g}_k$ as part of the model training

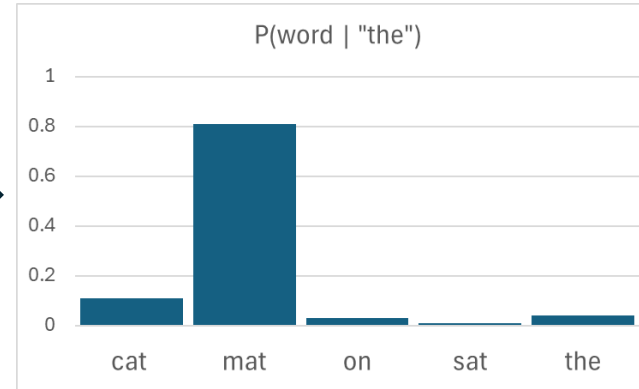
Training

- We have a model that calculates $\hat{P}(\text{word} \mid \text{preceding context} ; \underline{\theta})$
 $\underline{\theta}$ = Model parameters, e.g. query & key projection matrices, NN-parameters, etc.
- We have lots of training data – large text corpora from internet
- Minimize the cross-entropy:

$$\begin{aligned} & - \sum_i \sum_{w_i} P(w_i \mid \text{preceding context}) \log \hat{P}(w_i \mid \text{preceding context}) \\ & = - \sum_i \log \hat{P}(\text{Observed word at position } i \mid \text{preceding context}) \end{aligned}$$

Predicting the next word from context embeddings

the	cat	sat	on	the



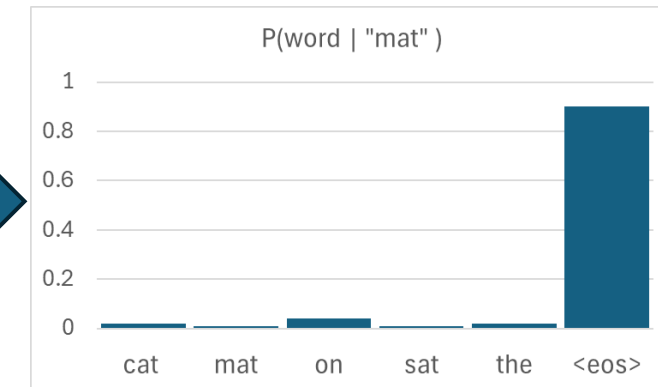
“mat”



cat	sat	on	the	mat

Transformer Model

cat	sat	on	the	mat



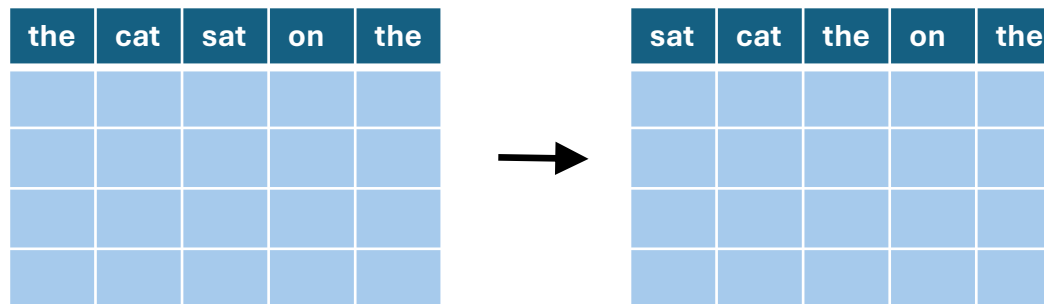
<eos>



Positional encoding

- Let's look at our formula for the attention probabilities

$$A_{n,n'} = \frac{\exp(\text{query}_n \cdot \text{key}_{n'})}{\sum_{n''} \exp(\text{query}_n \cdot \text{key}_{n''})}$$

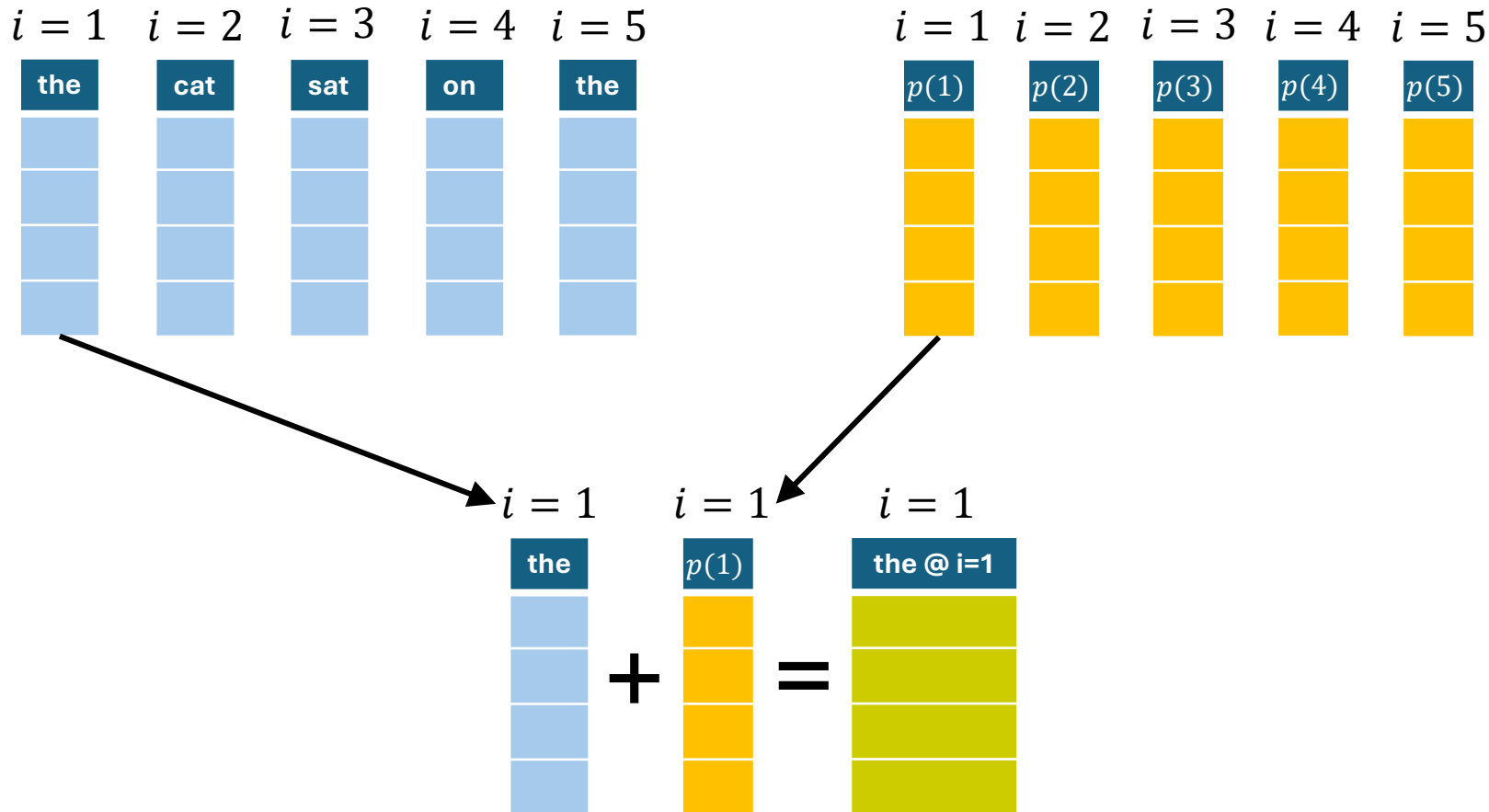


$A_{\text{on},\text{sat}}$ is unchanged

The input embeddings, and hence the context embeddings don't take into account the position of the token in the context (other than through masking) !!

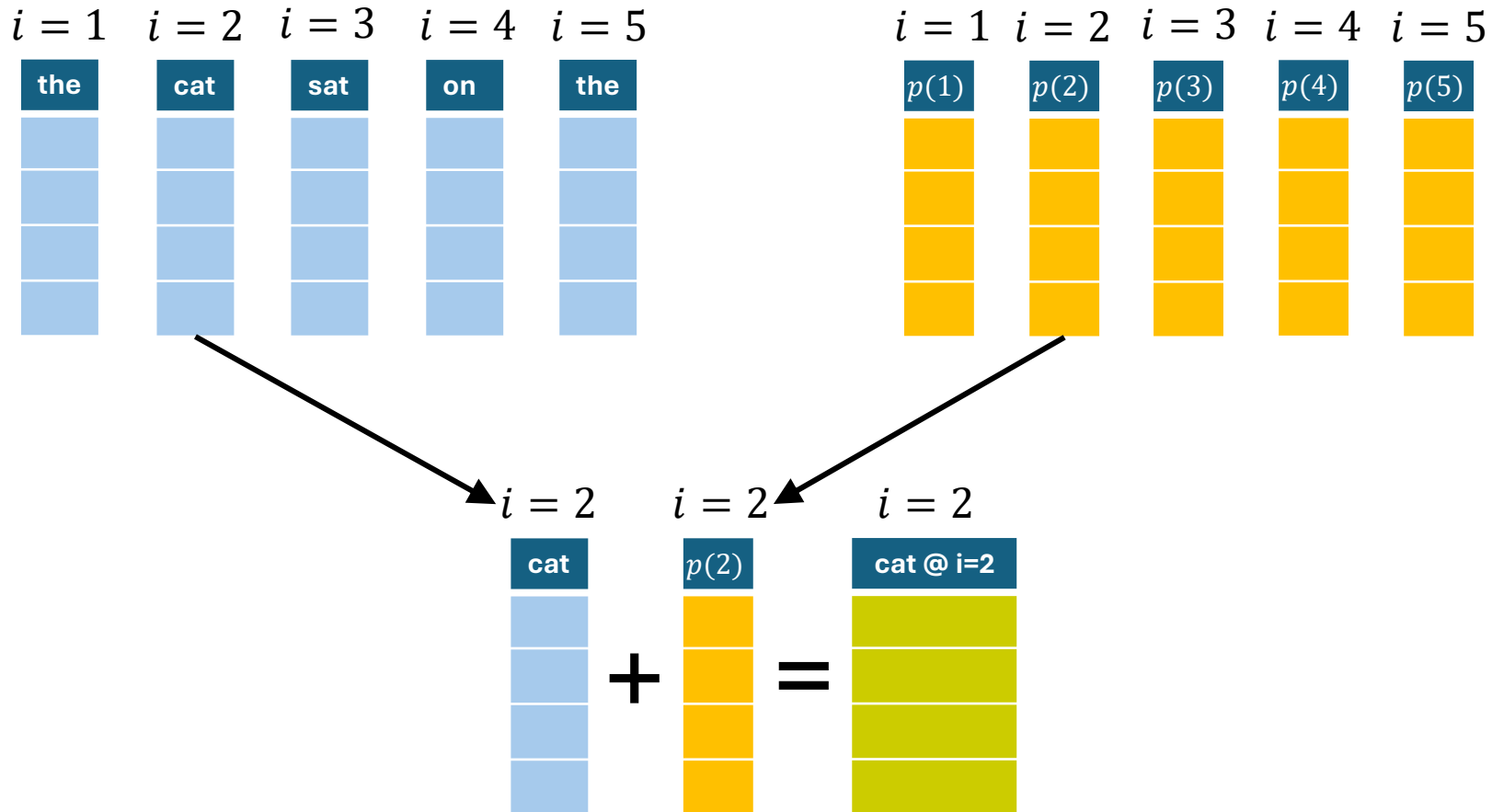
Positional encoding

- We solve by adding position dependent vectors to the input



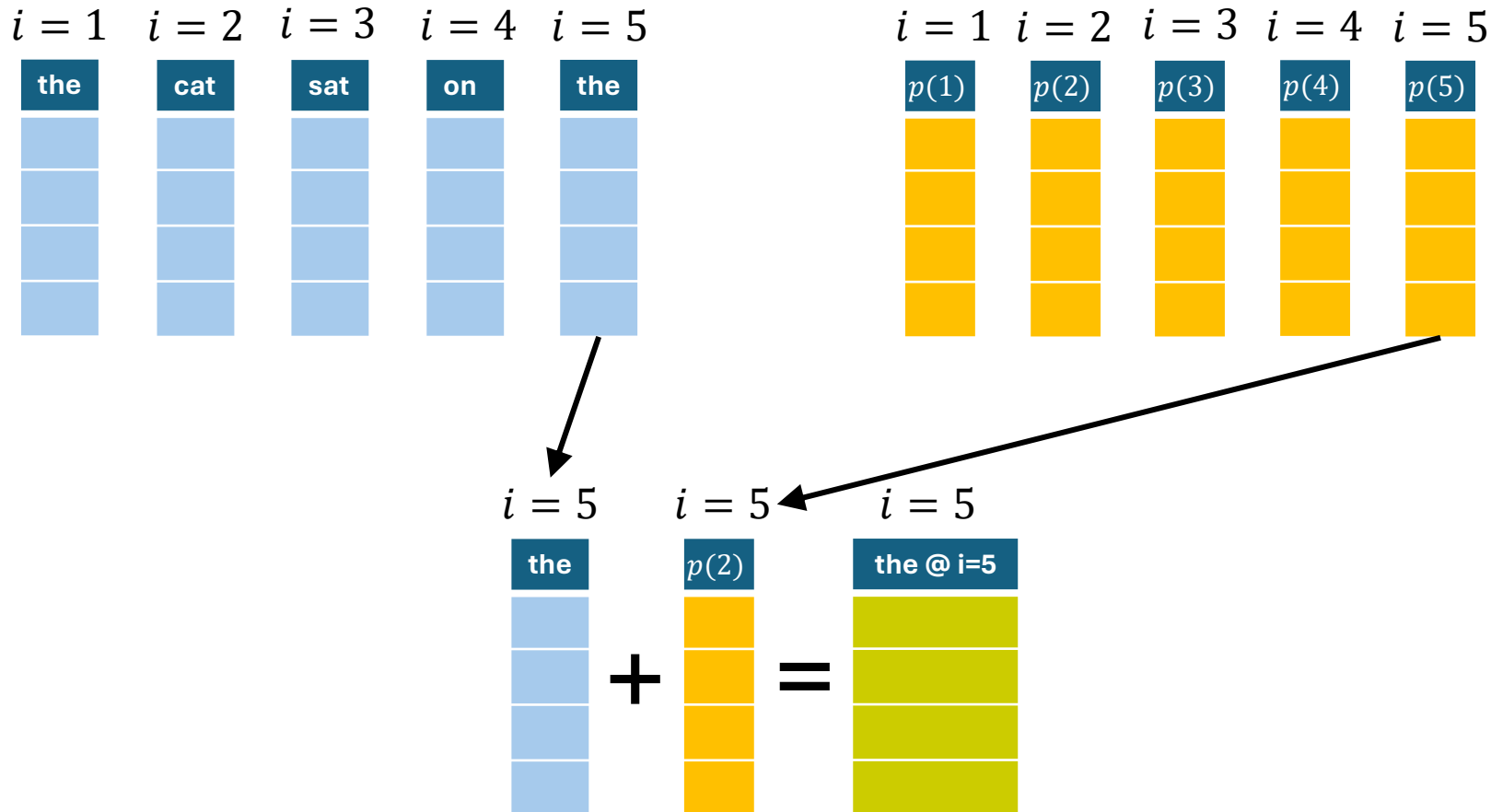
Positional encoding

- We solve by adding position dependent vectors to the input



Positional encoding

- We solve by adding position dependent vectors to the input



Positional encoding

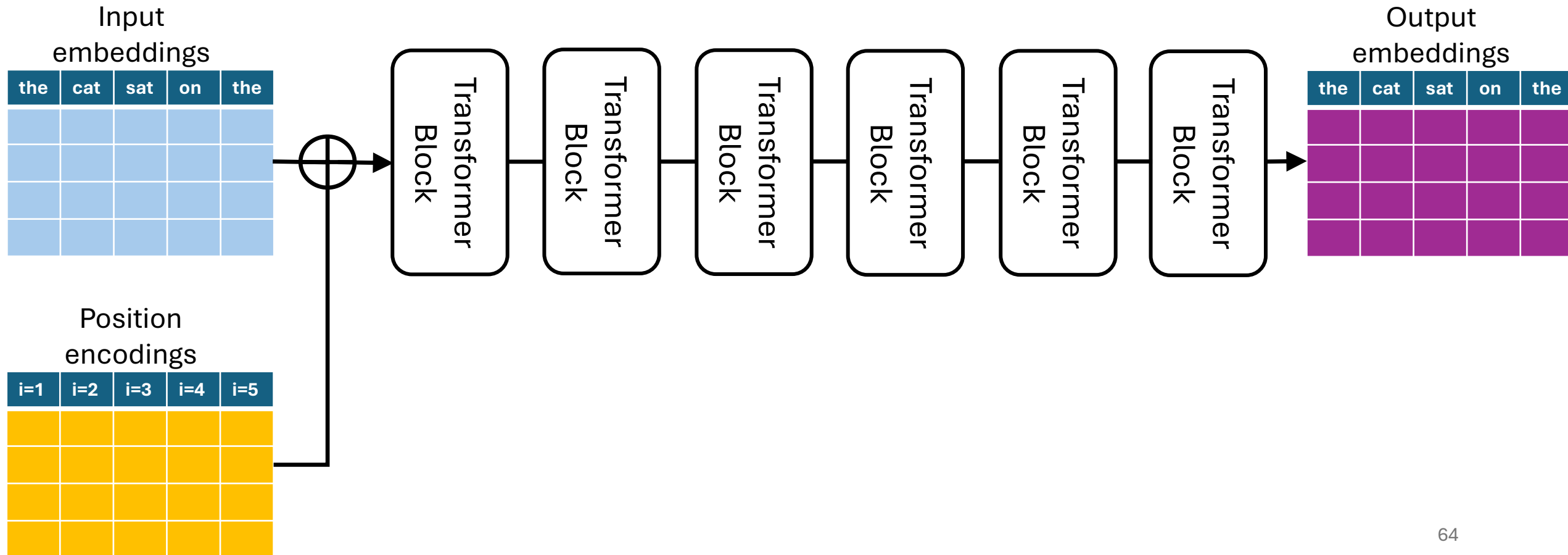
- Example functions

- $p_{2j}(i) = \sin\left(\frac{i}{10000^{2j/d}}\right)$
- $p_{2j+1}(i) = \cos\left(\frac{i}{10000^{2j/d}}\right)$

$$j = 1, 2, \dots, \lfloor d/2 \rfloor$$

Positional encoding

- Putting it all together (finally!)

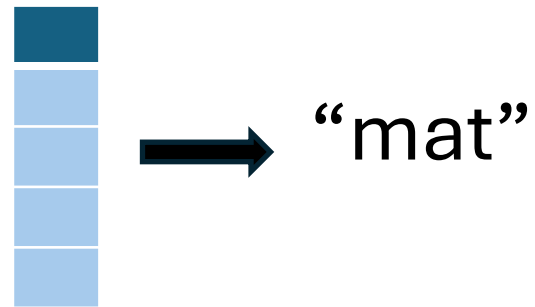


Different types of transformers

Encoders, Decoders, Cross-attention

Different types of transformers

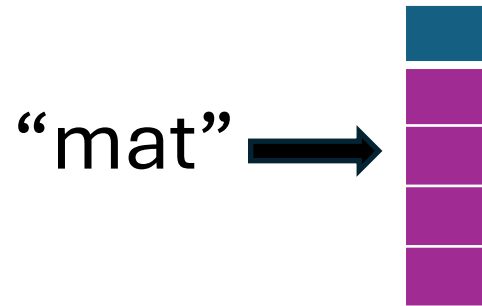
- What we covered was embedding vector → token (word)



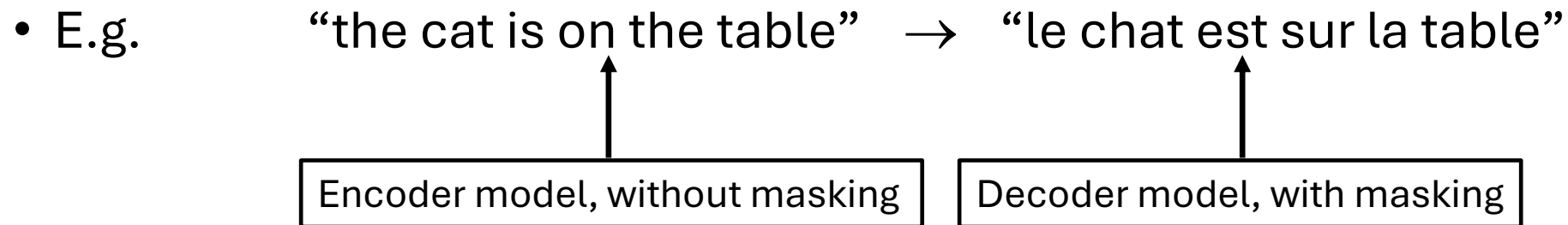
- We ‘decoded’ the embedding vector
- ChatGPT, Claude, etc are known as ‘decoder-only’ models

Different types of transformers

- We also have encoders: token $\rightarrow$ context embedding vector



- Encoders are useful where we have all the tokens (no prediction), we just need to map them to appropriate vectors



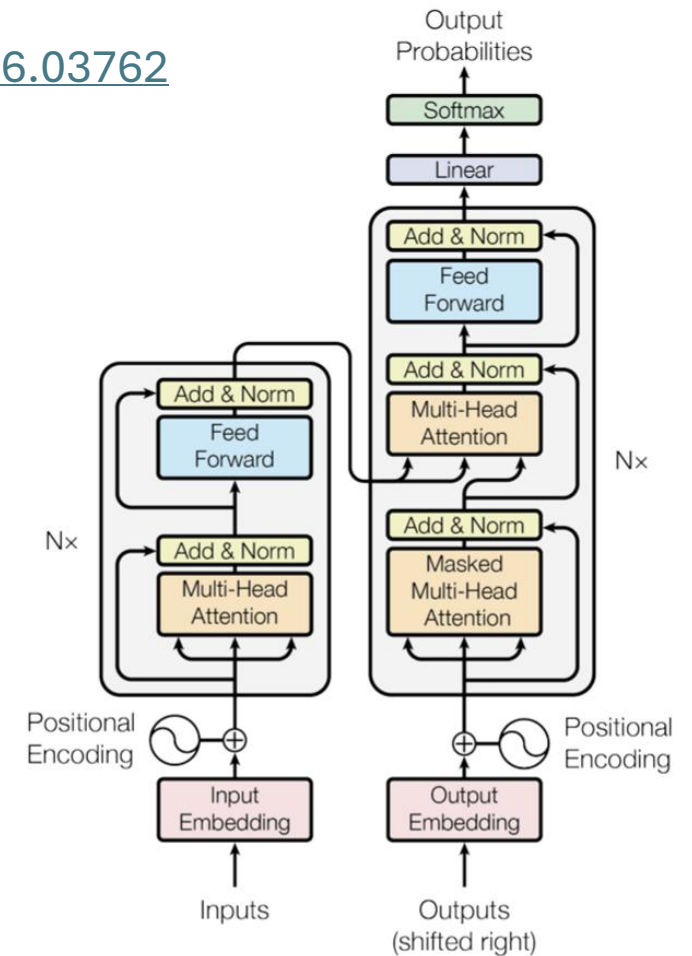
Different types of transformers

‘Attention is all you need’, Vaswani *et al*, 2017, [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)

“the cat is on the table” → “le chat est sur la table”

Encoder model, without masking

Decoder model, with masking



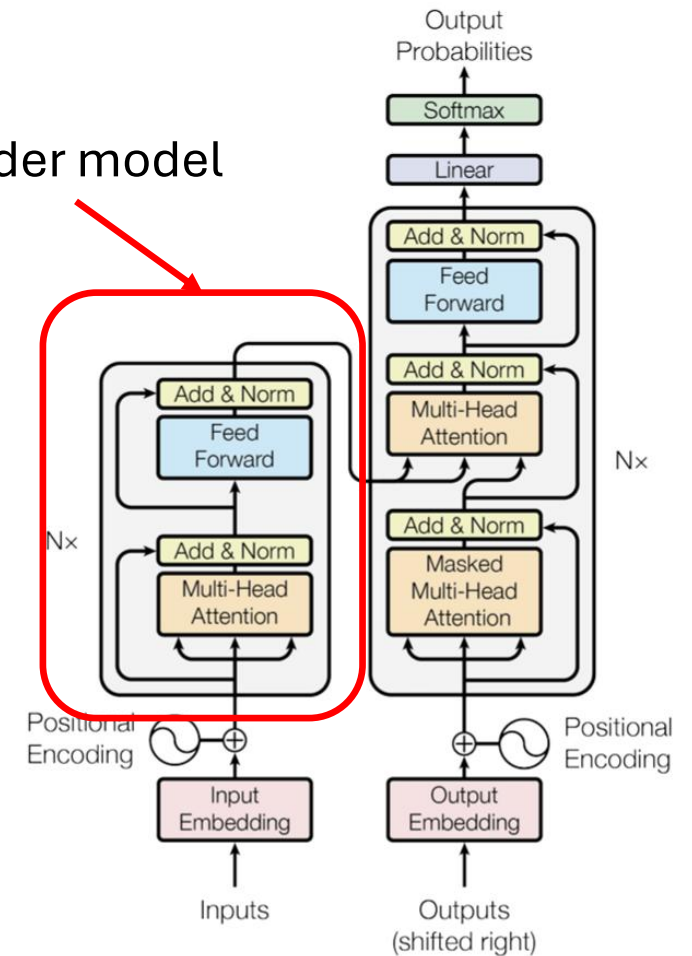
Different types of transformers

“the cat is on the table” → “le chat est sur la table”

Encoder model, without masking

Decoder model, with masking

Encoder model



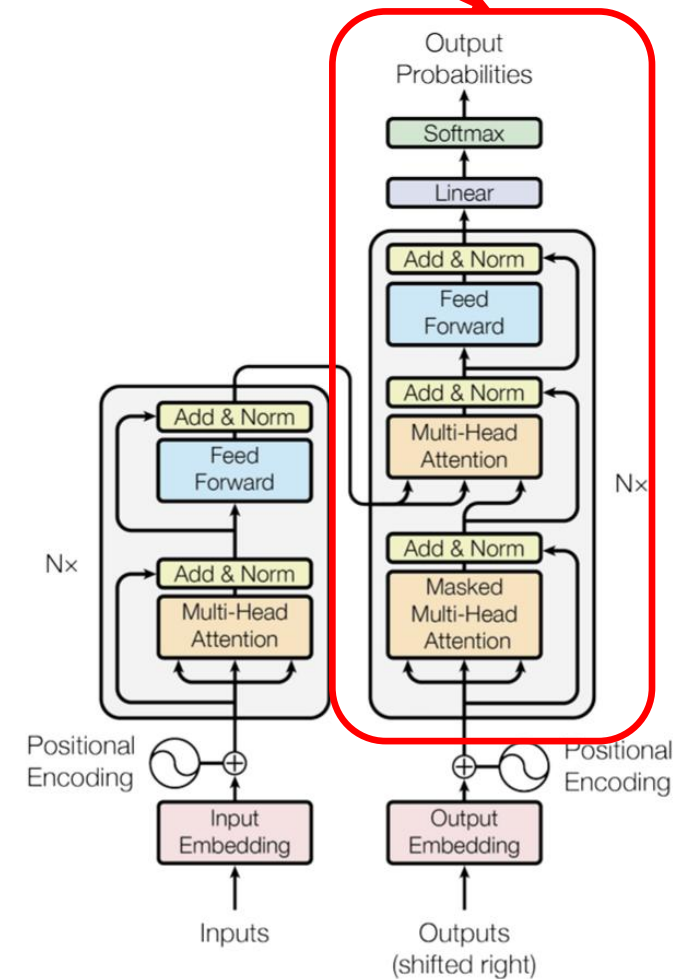
Different types of transformers

“the cat is on the table” → “le chat est sur la table”

Encoder model, without masking

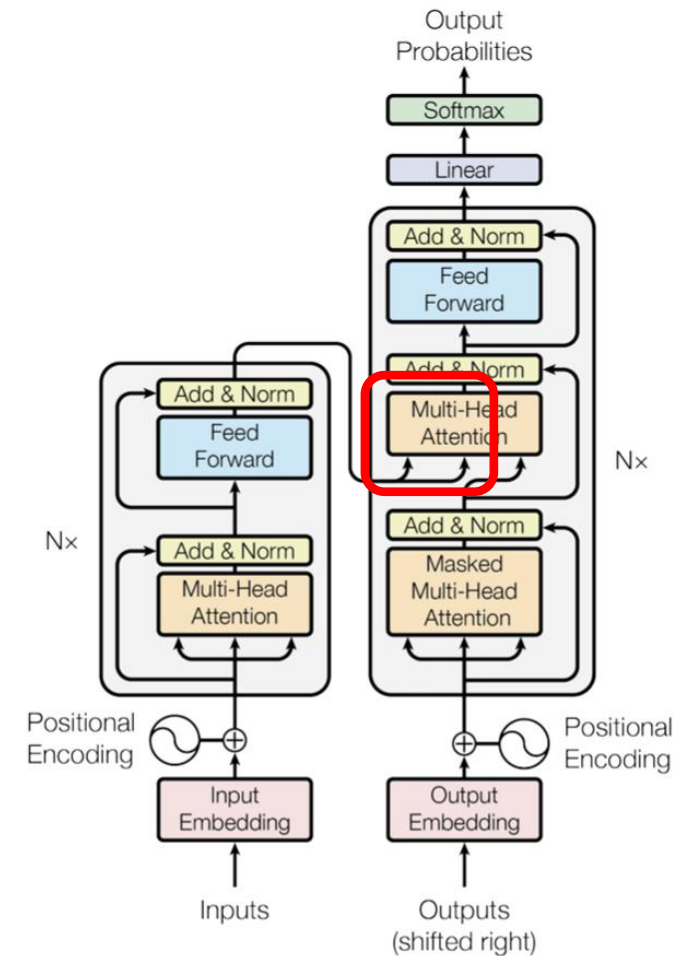
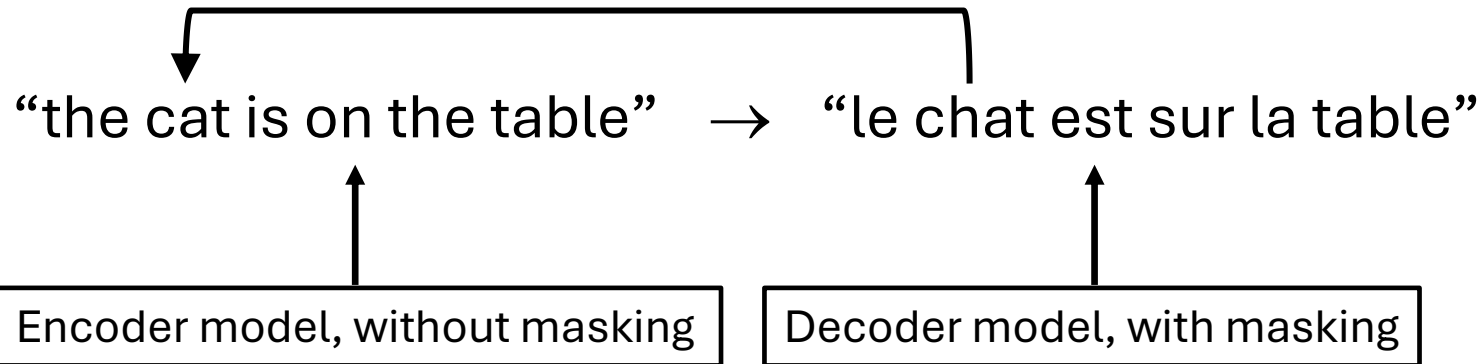
Decoder model, with masking

Decoder model



Different types of transformers

Cross attention



Code example

Using PyTorch to build transformer layers

Code Example 2

- Open up the Jupyter notebook ODSCEast2026_CodeExample2.ipynb in the github repository

https://github.com/dchoyle/ODSCEast2026_TransformerBootcamp/

Wrap up: What we learnt

- Next token prediction is a powerful capability
- To solve:
 - 1) We map very large vocabularies to lower dimensional vectors
 - 2) Each token in a sequence is initially represented by an embedding vector
 - 3) We transform each token to a new vector that also encodes the surrounding context
- The transformation model contains lots of parameters.
- We learn the model parameters by training on very large datasets
- Transformer models can be used to encode and/or decode tokens
- We can have applications that use only decoder blocks, only encoder blocks, or both.

Wrap up: What to learn next

- [The hundred page language models book](#) - Andriy Burkov
- [An introduction to Transformers](#) – R. Turner, arXiv:2304.10557
- [Decoder-Only Transformers: The Workhorse of Generative LLMs](#)
- Cameron R. Wolfe. Substack article
- [Attention is all you need](#) – Vaswani et al, arXiv:1706.03762
- [Paper Walkthrough: Attention Is All You Need](#)
- Muhammad Ardi. Towards Data Science article

Wrap up: What I haven't told you

- The engineering challenges, scale, and costs associated with foundational model training are significant
- Foundational models are general – but can be fine-tuned
 - Domain specific datasets
 - LoRA
- Problem of hallucinations can be mitigated but not eliminated
 - RAG
 - CoT

Thank you for listening Questions?

github.com/dchoyle/ODSCEast2026_TransformerBootcamp



www.linkedin.com/in/davidchoyle/



www.hoyleanalytics.org

